SkillNavora

MODULE 2 HANDBOOK

Module 2: Data Engineering, Databases and Data Integration

5 sections, every topic explained with how it works, examples and resources.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Module 2: Data Engineering, Databases and Data Integration

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

Sections

1. 2.1 Data Fundamentals
2. 2.2 SQL and PostgreSQL
3. 2.3 Database Technologies
4. 2.4 APIs and Data Integration
5. 2.5 Data Engineering

MODULE 2 · SECTION 2.1

2.1 Data Fundamentals

DEFINITION

> Data is recorded facts. Structured data sits in fixed tables; semi-structured data (JSON, XML) has tags but flexible shape; unstructured data (text, images, audio) has no fixed schema.

HOW IT WORKS

- Validation checks that values are the right type and range; quality checks look for duplicates, missing values and inconsistent units.
- Integrity rules (keys, constraints) stop invalid records from being saved.
- CSV and Excel are flat table formats; JSON nests objects and lists.

EVERY TOPIC EXPLAINED

Types of data: structured, semi-structured, unstructured. Structured data fits rows and columns with a fixed schema (customer table). Semi-structured data has labels but flexible shape (JSON API replies, XML, logs). Unstructured data has no schema (emails, PDFs, images, audio, call recordings) and is where LLMs add most value, because they can read it.

Formats: JSON, XML, CSV, Excel. CSV: plain text, one row per line, comma-separated; simple but no types. Excel (.xlsx): sheets, formulas and formatting. JSON: nested objects and arrays, the default for APIs. XML: tagged markup, common in older enterprise and government systems.

Data validation, quality and integrity. Validation checks each value against rules (email format, date range, required fields). Data quality covers completeness, accuracy, consistency, timeliness and uniqueness. Integrity is enforced by keys and constraints so references always point to real records. Bad data in means bad AI answers out.

WORKED EXAMPLE

> A lead record arrives as JSON with a nested address; you flatten it into two tables and reject rows whose email is missing.

RESOURCES

- pandas user guide (official) https://pandas.pydata.org/docs/user_guide/index.html
- JSON specification https://www.json.org/json-en.html

2.2 SQL and PostgreSQL

DEFINITION

> SQL is the language for querying relational databases. PostgreSQL is a free, widely used relational database that supports SQL standards and extensions.

HOW IT WORKS

- SELECT picks columns, WHERE filters rows, GROUP BY aggregates, ORDER BY sorts; JOIN combines tables on keys.
- CTEs (WITH) name intermediate results. Window functions (OVER) calculate across related rows without collapsing them, for example a running total.
- Indexes speed lookups; transactions (BEGIN/COMMIT) make multi-step changes all-or-nothing; normalisation removes duplicated data across tables.

EVERY TOPIC EXPLAINED

Databases, tables, rows, columns, primary and foreign keys. A relational database stores data in tables. Each row is one record; each column is one attribute. A primary key uniquely identifies each row (customer_id). A foreign key in another table points to it (orders.customer_id), linking the tables.

SELECT, WHERE, GROUP BY, ORDER BY, JOIN, subqueries, CTEs, window functions. SELECT name, city FROM customers WHERE city = 'Pune' ORDER BY name; GROUP BY city with COUNT(*) aggregates per group; JOIN orders ON orders.customer_id = customers.id combines tables; a subquery nests one query inside another; a CTE (WITH recent AS (...) SELECT ...) names a step for readability; window functions such as ROW_NUMBER() OVER (PARTITION BY city ORDER BY created_at) rank rows within groups.

Aggregate functions, indexing, optimisation, transactions, normalisation. Aggregate functions: COUNT, SUM, AVG, MIN, MAX. An index (CREATE INDEX ON orders(customer_id)) speeds lookups at a small cost to writes; EXPLAIN shows how a query runs. A transaction groups changes so they all succeed or all roll back. Normalisation splits data into related tables to avoid duplication and update errors.

PostgreSQL installation and setup. Install PostgreSQL locally (installer or Docker: docker run -e POSTGRES_PASSWORD=secret -p 5432:5432 postgres) or use a hosted service (Supabase, Neon, RDS, Cloud SQL). Connect with psql or a GUI (pgAdmin, DBeaver). Create a database and a user with only the permissions the app needs.

WORKED EXAMPLE

> SELECT customer_id, SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS running_total FROM orders; returns a running total per customer on every row.

RESOURCES

- PostgreSQL window functions tutorial (official) https://www.postgresql.org/docs/18/tutorial-window.html
- PostgreSQL SQL tutorial (official) https://www.postgresql.org/docs/current/tutorial.html

2.3 Database Technologies

DEFINITION

> Different databases suit different jobs: relational (PostgreSQL, MySQL) for structured records, document (MongoDB) for flexible JSON, key-value (Redis) for fast caches, and vector databases for similarity search.

HOW IT WORKS

- Relational databases enforce schema and relations; NoSQL stores trade some of that for flexibility and scale.
- Vector databases (Pinecone, Qdrant, pgvector) store embeddings and return the closest matches to a query vector.
- Supabase is hosted PostgreSQL with authentication and APIs added on top.

EVERY TOPIC EXPLAINED

PostgreSQL, Supabase, MySQL, MongoDB, Redis, SQLite. PostgreSQL: full-featured open-source relational database, the default choice. Supabase: hosted PostgreSQL plus auth, storage, APIs and vector search. MySQL: popular relational database for web apps. MongoDB: document database storing JSON-like records. Redis: in-memory key-value store for caching, sessions and queues. SQLite: a file-based database ideal for prototypes and local apps.

Vector stores: Pinecone, Qdrant, pgvector. Pinecone: fully managed vector database. Qdrant: open-source vector database you can self-host or use in the cloud. pgvector: a PostgreSQL extension that adds a vector column type and similarity search, so you can keep vectors next to your normal data.

Relational vs NoSQL vs vector databases. Relational: strict schema, joins and transactions; best for business records. NoSQL: flexible schema and horizontal scale; best for varied documents or very high volumes. Vector: similarity search over embeddings; best for semantic search and RAG. Many AI systems use all three.

WORKED EXAMPLE

> A support app keeps tickets in PostgreSQL, caches the top FAQs in Redis, and stores document embeddings in pgvector for retrieval.

RESOURCES

- Supabase docs https://supabase.com/docs
- pgvector (GitHub) https://github.com/pgvector/pgvector

2.4 APIs and Data Integration

DEFINITION

> An API is a defined way for one program to ask another for data or actions. REST uses HTTP methods on resource URLs; GraphQL lets the client ask for exactly the fields it needs.

HOW IT WORKS

- A request has a method (GET reads, POST creates, PUT/PATCH updates, DELETE removes), a URL, headers and sometimes a JSON body. The response has a status code (200 ok, 401 unauthorised, 429 rate limited, 500 server error).
- Authentication: API keys or OAuth tokens prove who is calling. Keep keys in environment variables, never in code.
- Pagination returns large lists in pages; rate limits cap calls per minute; webhooks push events to your URL when something changes.
- Postman or curl tests endpoints before you write code.

EVERY TOPIC EXPLAINED

What an API is; REST; GraphQL fundamentals; HTTP methods. An API (application programming interface) is a contract for how software requests data or actions from other software. REST models resources as URLs (/customers/42) and uses HTTP methods. GraphQL exposes one endpoint where the client sends a query listing the exact fields it wants. HTTP methods: GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE (remove).

Requests, responses, headers, authentication, API keys, OAuth, webhooks. A request has a method, URL, headers (Content-Type: application/json, Authorization: Bearer <token>) and body. A response has a status code (2xx success, 4xx client error, 5xx server error), headers and body. API keys identify the calling app; OAuth lets a user grant your app limited access without sharing their password. Webhooks are HTTP calls the other system makes to your URL when an event happens.

Rate limits, pagination, API documentation, Postman, API testing. Rate limits cap requests per period (HTTP 429 means slow down; retry with exponential backoff). Pagination returns results in pages using page/limit or a cursor. API documentation (often OpenAPI/Swagger) lists endpoints, parameters and examples. Postman lets you build, save and test requests and share collections with your team.

Third-party integration, error handling, API security. Third-party integration: read the docs, get credentials, test in Postman, then write a small client function with timeouts. Error handling: check status codes, retry only safe failures, log errors without secrets. API security: HTTPS only, keys in environment variables or a secrets manager, least-privilege scopes, input validation and verifying webhook signatures.

WORKED EXAMPLE

> A CRM webhook posts a new lead to your endpoint; your code validates it, saves it, and returns 200 so the CRM does not retry.

RESOURCES

- MDN: HTTP overview https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview
- Postman learning center https://learning.postman.com/docs/introduction/overview/

2.5 Data Engineering

DEFINITION

> Data engineering moves data from sources to places where it can be used reliably: ingestion, transformation, storage and serving.

HOW IT WORKS

- ETL transforms data before loading it into a warehouse; ELT loads raw data first and transforms it inside the warehouse.
- Batch processing runs on a schedule; stream processing handles events as they arrive.
- Orchestrators (Airflow, Dagster, n8n) run steps in order and retry failures. Governance records who owns data and who may see it.

EVERY TOPIC EXPLAINED

Data pipelines, ETL vs ELT, ingestion, transformation, cleaning, validation. A data pipeline moves data automatically from sources to destinations. ETL = extract, transform, load (clean before loading). ELT = extract, load, transform (load raw, clean inside the warehouse). Ingestion collects data; transformation reshapes it; cleaning fixes errors and duplicates; validation checks it meets rules before use.

Batch and stream processing; orchestration. Batch processing handles data in scheduled chunks (nightly sales report). Stream processing handles each event as it arrives (fraud alert in seconds) using tools such as Kafka. Orchestration (Airflow, Dagster, Prefect, n8n) schedules steps, manages dependencies, retries failures and alerts on errors.

Data warehouses, data lakes, lakehouses, data governance. A data warehouse (BigQuery, Snowflake, Redshift) stores cleaned, structured data for analytics. A data lake stores raw files of any type cheaply (S3, Azure Data Lake). A lakehouse (Databricks, Iceberg) combines both. Data governance defines ownership, access, quality standards, lineage and retention.

WORKED EXAMPLE

> Every night a job pulls yesterday's orders from the API, cleans the currency fields, and loads them into a warehouse table that the dashboard reads.

RESOURCES

- Apache Airflow docs https://airflow.apache.org/docs/
- dbt docs (transformation) https://docs.getdbt.com/